Algebraic World Model: From Exploration to Proofs

Anonymous authors
Paper under double-blind review

Abstract

AI has made remarkable progress in mathematics, solving challenging problems with machine-checkable proofs in Lean, yet typically relying on predefined mathematical definitions and formal specifications. This raises a natural question: can it reason rigorously about an unknown world? We introduce the Algebraic World Model (AWM), a framework that turns interaction with unfamiliar environments into explicit mathematical models supporting formal reasoning. AWM explores the environment and develops executable Python hypotheses, continually testing and revising them against observations. It then structures these hypotheses into typed objects and shared rules, simplifies their structure through category-theory-inspired transformations, and formalizes the resulting model in Lean 4 to prove reachability and impossibility.

AWM enables formal proofs of impossibility beyond explored trajectories. On two impossible game levels, it constructs closed, kernel-checked Lean proofs excluding every legal winning sequence under its learned models in all six runs, while Claude Opus 5.5 and GPT-6 Astra produce none in twelve baseline runs. Four of AWM's six initial bounds match the engine optima, compared with one for Opus 5.5 and none for GPT-6 Astra; feedback-guided refinement brings AWM to six of six. Refinement works because each proof names the rules it rests on: these rules alone locate a hidden error in 29 of 42 runs, nearly twice as often as per-rule evidence.

Its models make individual state transitions explicit and checkable. Across all 25 public ARC-AGI-3 games, Lean replays the winning trajectories of all 183 levels against AWM's models, and AWM raises Opus 5 to a score of 99.9 with solutions that use 43% of the human baseline's actions.

1 Introduction

Artificial intelligence has made remarkable progress in mathematical reasoning: automated systems now produce Lean proofs of olympiad problems and of open Erdős problems (Achim et al., 2025; Hubert et al., 2026; Sothanaphan, 2026; Tsoukalas et al., 2026). These advances invite a broader question:

Can an agent reason rigorously about an unknown world?

In formal mathematics the definitions needed to state a problem are given in advance, and a proof checker verifies whether a conclusion follows from them. In an unfamiliar environment the agent must first discover the relevant objects, infer how actions transform them and determine what counts as success; the rules required for formal reasoning are themselves hypotheses about the world, learned from experience and open to error.

Euler's treatment of the Königsberg bridges shows what such a structure can provide (Euler, 1741). Land masses become vertices and bridges become edges. A walk crossing every bridge exactly once pairs arrivals with departures at each intermediate vertex, so the graph can have at most two odd-degree vertices; the Königsberg graph has four. The graph rules out every candidate walk, including the walks nobody tried. Finding one winning route shows that a goal can be reached; showing that none exists needs a model of this kind, and in an unfamiliar environment that model has to be learned.

The Algebraic World Model (AWM) builds such a model through three stages (Figure 1). **Explore & Prototype:** the agent actively explores the environment, building and iteratively revising a provi-

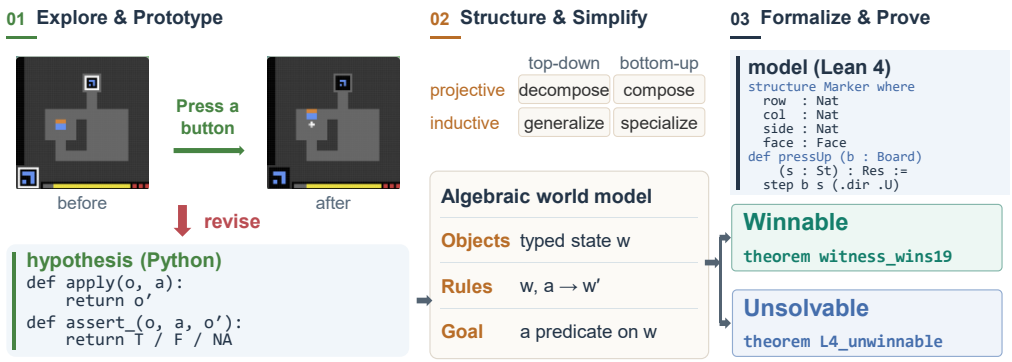


Figure 1: AWM's three-stage pipeline: interactive exploration builds a provisional executable model, algebraic transformations reveal and simplify its structure, and Lean 4 formalization enables certified proofs of reachability or impossibility.

sional executable model from observed transitions. **Structure & Simplify:** it organizes this prototype into typed objects and rules, then applies category-theory-inspired transformations to discover and simplify their underlying structure while preserving empirically tested distinctions. **Formalize & Prove:** it encodes the model in Lean 4 (de Moura & Ullrich, 2021), and constructs proofs of reachability, impossibility, and other stated properties, checked by Lean's small trusted kernel. A winning witness certifies reachability; an invariant or a suitable cost lower bound, can exclude every legal winning sequence under the model.

AWM can formally prove that a task is impossible. We evaluate this ability on ARC-AGI-3 (ARC Prize Foundation, 2026b), whose games reveal neither their rules nor their goals, by setting the action budgets of two `sc25` levels just below their minimum winning costs (§4). In all six runs, AWM proves in Lean that no legal winning sequence exists under its learned model, while Claude Opus 5.5 and GPT-6 Astra produce no such proof in twelve runs. On one level, Astra judges the level impossible but puts the minimum winning cost at 22, whereas AWM proves the engine-optimal bound of 19. When an AWM bound is wrong, the proof points to the error: the rules it rests on locate a hidden error in 29 of 42 runs, nearly as often as being told which rule is wrong.

AWM's models can be read and checked step by step. Across all 25 public ARC-AGI-3 games, AWM builds models with a median of six object types and ten rules, and Lean replays the winning trajectories of all 183 levels against them (§3).

We make three contributions.

- **From exploration to algebraic structure.** AWM turns interaction with an unknown environment into executable hypotheses, then discovers shared mathematical structure through category-theory-inspired transformations. The resulting compact model expresses related mechanisms as reusable, compositional rules, enabling unified reasoning about their behavior (§2).
- **Formal impossibility reasoning in unknown worlds.** AWM constructs closed, kernel-checked Lean proofs that rule out every legal winning sequence under its learned model and specified constraints, including sequences never explored. This moves beyond empirical judgments based on finite play to formally certified conclusions under explicit learned premises (§4).
- **Inspectable models and traceable proofs.** AWM represents environmental dynamics through explicit typed objects and named rules, whose predictions can be checked against observed transitions. Its formal representation supports step-by-step replay and exposes the premises behind each proof, making learned mechanisms interpretable and discrepancies traceable to their underlying assumptions (§3; §4.3).

2 Method

AWM aims to turn interaction with an unknown environment into a mathematical model that supports rigorous reasoning about both reachability and impossibility. This requires more than collecting rules

that explain observed transitions: the rules must form a coherent representation of the environment, and that representation must support reasoning about action sequences the agent has never explored.

We first describe the target model (§2.1), then present the three stages that construct and use it (Figure 1): Explore & Prototype develops executable hypotheses through interaction; Structure & Simplify organizes them into a compact algebraic model; and Formalize & Prove completes the model in Lean 4 and establishes machine-checked results.

2.1 The Algebraic World Model

An algebraic world model describes an environment through typed objects, rules, and a goal predicate, represented in executable Python and formal Lean 4 to support both empirical testing and mathematical reasoning.

Objects. Object types define the structure and validity of states, providing a common vocabulary for describing the environment. A parser maps observations, together with any required history or auxiliary information, to typed states w.

Rules. Rules describe how actions transform these states. Each Python rule specifies an applicability scope and a checker that reports agreement, contradiction, or non-applicability on observed transitions; effect rules also predict successor states or their components. Together, the rules define legal actions, their effects and costs $\text{cost}(w, a)$, while the goal predicate identifies successful states.

Related mechanisms can share a parameterized operation $r(\theta)$, with θ determined by the state. This allows common properties to be expressed and proved uniformly across different mechanisms.

Formal representation. The same model is formalized in Lean 4, with objects and rules encoded as definitions and properties of their dynamics expressed as theorems. Its executable representation supports testing against observations, while its formal representation supports kernel-checked proofs over legal action sequences (§2.4).

2.2 Explore & Prototype

Setting. We consider deterministic, goal-directed environments with partial observations, modeled in the POMDP setting (Kaelbling et al., 1998). At step t, the agent observes o_t, takes an action a_t, and receives o_{t+1} without access to the underlying state or transition function. Its experience forms a corpus $\mathcal{D}_t$ of distinct transitions $x = (o, a, o')$.

Building and testing hypotheses. AWM actively explores the environment and maintains a provisional executable model as a *concept book*. Each concept is a Python hypothesis about an environmental mechanism, containing a natural-language claim, an applicability scope, supporting transitions, a checker, and an optional predictor.

For a concept c, its checker returns $v_c(x)$: `TRUE` if the transition agrees with the hypothesis, `FALSE` if it contradicts it, and `NA` if it falls outside its scope. The optional predictor specifies expected action effects for comparison with subsequent observations.

After each action batch, AWM rechecks the concept book $\mathcal{C}_t$ against the expanded corpus $\mathcal{D}_t$. Further interaction is allowed only when

$$\forall c \in \mathcal{C}_t,\ \forall x \in \mathcal{D}_t : \quad v_c(x) \notin \{\mathsf{FALSE}, \mathsf{ERR}\}, \tag{1}$$

where ERR denotes checker failure, including exceptions and timeouts.

This recheck ensures that contradicted or failing concepts must be revised or withdrawn before exploration continues. The agent then keeps revises and uses its hypotheses to plan toward inferred goals or probe competing explanations.

The resulting prototype captures tested mechanisms but may describe related behaviors independently, motivating the next stage.

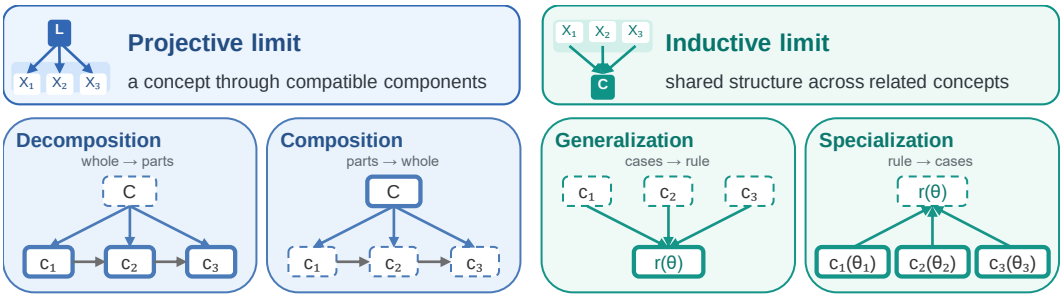


Figure 2: Four transformations inspired by projective and inductive limits, applied to object types and rules. Dashed boxes mark removed components and bold boxes mark introduced components; arrows indicate relationships between components. Generalization uses $r(\theta)$, and an instance is written $r(\theta_i)$; specialization makes distinctions explicit through case-specific types or parameters.

2.3 Structure & Simplify

The concept book may contain fragmented, overlapping descriptions of environmental mechanisms. Structure & Simplify turns these hypotheses into a compact model by identifying shared structure and reorganizing components through their relationships.

AWM first extracts the required object types, constructs a parser from observations to typed states, and rewrites concepts as rules over these objects. It then restructures these *components*—object types and rules—through four category-theory-inspired transformations. A *projective limit* assembles compatible descriptions into a coherent whole, while an *inductive limit* glues objects along shared structure (Mac Lane, 1998; Kashiwara & Schapira, 2006; Yuan, 2023). These two perspectives motivate transformations based on compatibility and shared structure, respectively (Figure 2).

Following projective limits, *decomposition* expresses a complex component through simpler components and their compatibility relations. Conversely, *composition* expresses a component through compatible existing components, moving the composite into the derived layer when a separate primitive definition is unnecessary.

Following inductive limits, *generalization* extracts shared structure from multiple components, expressing source rules c_i through a parameterized operation $r(\theta_i)$ with state-dependent θ_i. Conversely, *specialization* refines overly general components into more specific cases or introduces parameters to reduce exceptions and redundancy.

Preserving tested distinctions. Every transformation must preserve verdicts on recorded transitions. For rules, AWM also uses a diagnostic set $\mathcal{N}$ of perturbed recorded transitions to check whether simplification loses previously detected rejections.

Define $\text{Rej}(c) = \{x \in \mathcal{N} : v_c(x) = \textsf{FALSE}\}$ and $\text{Rej}(S) = \bigcup_{c \in S} \text{Rej}(c)$. Replacing rules S with T must preserve coverage of recorded transitions and satisfy

$$\text{Rej}(T) \supseteq \text{Rej}(S). \tag{2}$$

For object types, which have no checkers, transformations are evaluated by distinctions between recorded states, using state pairs as witnesses, and by rule verdicts once rules are available.

AWM applies these transformations and tests to object types, then rules, and finally rechecks object types against the settled rules. The resulting compact structure provides a common representation for formal reasoning.

2.4 Formalize & Prove

AWM completes the model in Lean 4, encoding object types as structures or inductive types, rules as `def` definitions, and derived properties as `theorem` statements. The transition function dispatches to named rules, while unresolved assumptions remain explicit theorem hypotheses rather

than unchecked `axiom` declarations. Replaying recorded winning trajectories, the Lean model reproduces observed behavior on every component it represents, grounding its formal definitions in the dynamics the agent encountered. This enables theorems about all legal action sequences under the learned model, extending formal reasoning beyond the trajectories actually explored.

Proving reachability and impossibility. A level is *winnable* in the model if a legal action sequence reaches the goal. AWM proves reachability by constructing a witness checked in Lean. Impossibility can arise because the goal is unreachable, or because reaching it requires more cost than the available budget. AWM proves the former using an invariant that excludes the goal, and the latter using a lower bound on the cost of any winning sequence. For budgeted tasks, a potential V estimates the minimum remaining cost from each state and satisfies

$$V(w) \leq \text{cost}(w, a) + V(w') \quad \text{for every legal transition,} \qquad V(w) \leq 0 \quad \text{on the goal,} \tag{3}$$

so every winning sequence costs at least $V(w_0)$, proving impossibility when this exceeds the budget. All proofs are closed and checked by the Lean kernel, without unproved placeholders (`sorry`) or kernel-bypassing computation (`native_decide`), establishing rigorous guarantees under the learned model's explicit premises.

2.5 Example: one rule for four mechanisms

In game `lp85`, buttons move colored tiles, and a level is won when every slot holds its matching color. During Explore & Prototype, the agent discovers eleven concepts describing four button mechanisms: belts, stars, hubs, and panels (Figure 3a). In Structure & Simplify, AWM applies the generalization transformation to recognize that all four mechanisms permute tile positions, replacing the eleven concepts with a shared operation, `press_permutes`, parameterized by the pressed button. On the diagnostic set used for this merge, the shared rule retains all 55 rejections of the source concepts and rejects 21 additional corrupted transitions. The shared rule also shortens reasoning: because one color-count invariant covers every button, proofs that a variant is unsolvable need two to three times fewer lemmas than proofs over the eleven separate concepts (Appendix A).

In Formalize & Prove, Lean certifies a five-click winning sequence (Figure 3b) and proves a recolored variant unwinnable using the color-count invariant shared by all button permutations (Figure 3c). The final model contains six object types and six rules, and Lean replays all 199 recorded transitions without mismatch.

3 Twenty-five public games

We run AWM with Claude Opus 5 on all 25 public ARC-AGI-3 games (183 levels), starting each game from scratch, and examine the models it builds, how far Lean can check them, and whether they generalize beyond the play that produced them.

Compact models that reproduce observed behavior. Across the 25 games, AWM writes 742 executable concepts, each checked against every recorded transition, and withdraws the 143 that a later observation refutes. Structure & Simplify organizes the rest into 170 object types and 283 rules, a median of 6 types and 10 rules per game. Shared patterns include translations, permutations and goal conditions. Lean replays the winning trajectory of every one of the 183 levels from its initial state and verifies its final state against the modeled goal (Table 2).

Generalization beyond observed transitions. We next test the learned rules on behavior the record never showed. We freeze the models of `lp85` and `ft09` after exploration and evaluate them on transitions from random play with previously unseen screens (seeds 0–9; detailed results in the supplementary material). The final frozen rules accept every valid transition in the new-trajectory sets. On the two modified `lp85` layout sets, they predict the next tile layout in all 4,160 cases; on the two changed-mechanic `ft09` sets, they reject all 4,031 transitions. Corruption results are reported separately in the supplement.

The changed mechanic modifies `ft09`'s hidden click pattern, causing each click to change 110 or 146 cells instead of 38; rejecting these transitions shows that the rules detect behavior inconsistent with the learned dynamics.

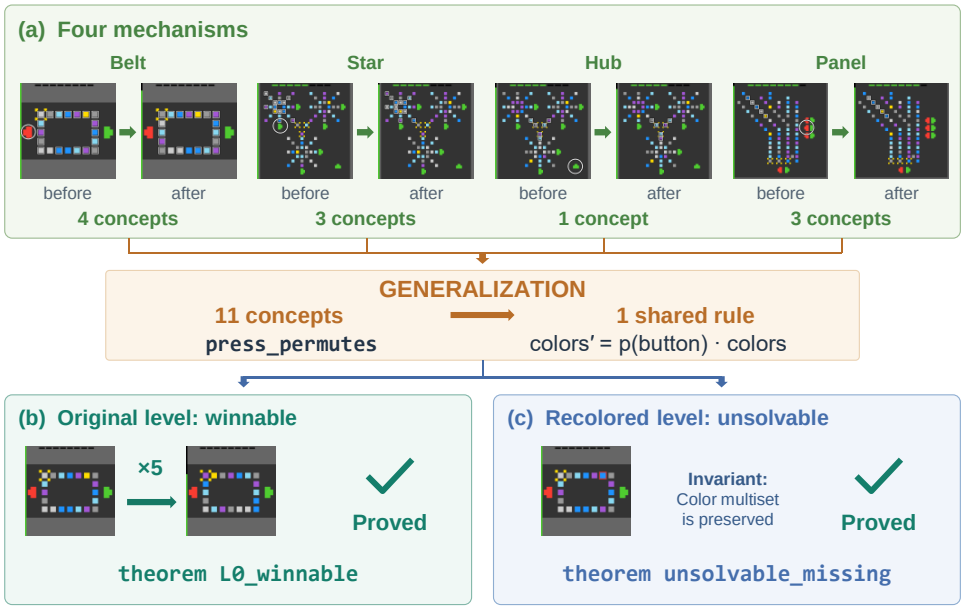


Figure 3: From shared structure to formal proofs in `lp85`. (a) Eleven concepts describing four button mechanisms are generalized into a shared permutation rule. Each before/after pair shows a recorded press, with the pressed button circled and changed tiles outlined. (b) A five-click sequence witnesses reachability. (c) Recoloring the only yellow tile makes the goal unreachable, since every press preserves color counts. Both conclusions are formally proved in Lean 4.

Table 1: Official scores on the 25 public ARC-AGI-3 games. Claude Opus 5 and GPT-6 Astra run under the official standard harness, which provides no code execution; the other systems execute code. AWM's certified clears follow uncertified sessions of the same run in 22 games. baseline1's actions are from its ARC Prize scorecard and Memento 3's as reported in (Zhao et al., 2026).

System	Code	Mean score	Games at 100	Certified actions	Actions / human
Claude Opus 5	no	40.7	5	—	—
GPT-6 Astra	no	68.3	10	—	—
GPT-6 Astra (adapter) (ARC Prize Foundation, 2026a)	yes	100.0	25	—	—
baseline1 (Rodionov, 2026b)	yes	99.0	23	8,347	0.49
Memento 3 (Zhao et al., 2026)	yes	100.0	25	7,518	0.44
AWM (ours)	yes	99.9	23	**7,448**	**0.43**

Table 2: Formalization and verification of the 25 learned world models in Lean 4. Axioms are those Lean's own logic uses; no proof adds axioms of its own.

Levels replayed	Rules formalized in Lean	Theorems	Axioms
183 / 183	220	1,374	only `propext`, `Classical.choice`, `Quot.sound`

Clearing the games with efficient solutions. The same models are effective for play. AWM with Opus 5 achieves a score of 99.9, compared with 40.7 for Opus 5 under the official standard harness without code execution. It scores 100 on 23 games and at least 97.6 on every game, performing competitively with leading code-executing systems (Table 1). Its certified clears require 7,448 actions, only 43% of the human baseline's 17,135 and the fewest among systems reporting this metric, compared with 7,518 for Memento 3 (Zhao et al., 2026) and 8,347 for baseline1 (Rodionov, 2026b). Including exploration and other sessions, AWM uses 24,604 engine actions in total (Appendix E). These results show that AWM can construct and use explicit world models while retaining com-

Table 3: Impossibility reasoning across six runs per agent. Bound accuracy is evaluated against engine optima before and after feedback-guided repair. First claim averages the earliest impossibility claim among runs making one, including later withdrawals; it is not proof-completion time.

Agent	Kernel-checked model proofs	Bounds matching optimum initial → feedback	Ever claimed impossible	First claim (attempt)
Claude Opus 5.5	0/6	1/6 → 1/6	3/6	5.3
GPT-6 Astra	0/6	0/6 → 2/6	2/6	8.0
AWM	**6/6**	**4/6 → 6/6**	**6/6**	**2.0**

petitive solution efficiency. §4 next examines whether such models also support formal proofs of impossibility.

4 Proving impossibility

A winning trajectory establishes reachability; impossibility requires excluding every legal winning sequence within the budget, including those never explored. We test whether AWM's learned algebraic models enable such exhaustive proofs, and whether those proofs hold for the game itself.

4.1 The game, the seal, and the baselines

In `sc25`, a block moves on a grid and casts spells through a 3×3 panel to reach the exit within the level's budget. Moves and panel clicks cost one unit, completing a spell one more, fuel refunds ten, and the winning move is free. We construct two variants of levels 4 and 5 by rearranging selected elements while preserving the game mechanics, then seal them at budgets of 18 and 35. Exhaustive engine search over all four openings and all 13 actions establishes minimum winning costs of 19 and 36, respectively: neither sealed level can be won, while one additional unit permits a win.

Claude Opus 5.5 and GPT-6 Astra receive the game interface and coding tools, including simulator construction and search; AWM uses Opus 5.5 with the procedure of §2 and a generic Lean skeleton. Each condition has three runs per level, with up to eight reset-delimited attempts per run, ending at game over or recorded abandonment. After each attempt, every agent is asked whether the sealed level can be won and to justify its verdict; AWM constructs Lean proofs under its learned model. Impossibility can be established by proving that the minimum winning cost exceeds the budget. We evaluate proof validity under the learned model separately from numerical agreement with the engine.

4.2 Certified lower bounds

Proving beyond explored trajectories. All six AWM runs produce closed, kernel-checked Lean proofs of impossibility under their learned models and explicit premises, while none of the twelve baseline runs produces such a proof (Table 3). Crucially, these proofs exclude all legal winning sequences within the budget under the learned models, including those never explored. For example, an AWM run for level 4 proves

$$\forall B \leq 18,\ \forall \mathbf{a}, \quad \mathrm{Won}(\mathrm{run}_B(w_0, \mathbf{a})) = \mathsf{false}, \tag{4}$$

and verifies a winning witness at $B = 19$. Together, the lower bound and witness certify a tight threshold under the model, whereas finding a winning route alone cannot exclude cheaper alternatives.

AWM establishes such bounds through potential or closure certificates over arbitrary legal action lists. Certificate data may come from any search; Lean verifies it, so the optimality claim never rests on trusting the search, and Lean also rejects vacuous premises, reversed bounds and drift between the Python and Lean versions of a model, which a Python checker lets through (Appendix C). Because each proof names its rules and premises, it can be checked both as a proof and, rule by rule, against the game.

Beyond unproved numerical judgments. Only five of twelve baseline runs claim impossibility, four with incorrect bounds. Even the sole correct Claude Opus 5.5 bound lists "that my hand-counted

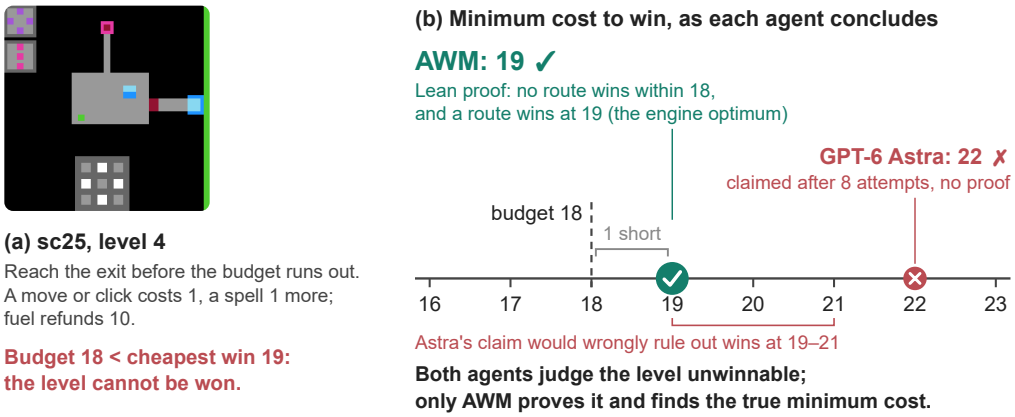


Figure 4: Sealed sc25 level 4. AWM proves in Lean that the minimum winning cost is 19, one unit above the budget, and finds a route at that cost; GPT-6 Astra claims 22 without a proof.

path lengths are truly minimal" as *assumed, not checked*. Finding a winning route establishes an upper bound on the minimum cost, not its optimality. AWM instead provides kernel-checked lower-bound proofs under its learned models.

Figure 4 illustrates this distinction on L4. Astra finds a winning route costing 22 and assumes it is optimal without proof, incorrectly ruling out wins at budgets 19–21. In contrast, AWM formally proves a lower bound of 19 under its learned model and verifies a matching winning witness, establishing a tight bound that agrees with the engine optimum.

Proof validity and bound accuracy are distinct. Before feedback, four of six AWM bounds agree with the engine optima, compared with one for Opus 5.5 and none for GPT-6 Astra.

Some runs identify a correct winning sequence but charge the otherwise free winning move, reporting a bound one unit higher. Because this cost cannot be observed on sealed levels, we count such bounds as correct for all agents only when the discrepancy is explicitly attributable to this accounting convention, not to an incorrect route or cost estimate.

The remaining two AWM bounds, 20 on L4 and 47 on L5, arise from overly restrictive learned rules. Their proofs remain valid under those models, although their numerical thresholds disagree with the game.

4.3 Auditing premises and proving again

After the initial evaluation, the two AWM runs with bounds of 20 and 47 are told only that the engine permits a cheaper win. AWM revisits its explicit rules, tests alternative formulations through interaction, and constructs new proofs under the refined models.

On L4, AWM discovers that growing onto fuel collects it, reducing the bound from 20 to 19. On L5, it identifies terrain rather than limited range as the cause of a failed shot, reducing the bound from 47 to the engine optimum of 36. Both refinements succeed within four additional attempts, each yielding a new closed Lean proof. With this feedback, AWM reaches correct bounds in all six runs.

We provide the same discrepancy feedback and additional attempt budget to the four baseline runs with incorrect bounds. Astra corrects both bounds but produces no formal proofs, while Claude Opus 5.5 corrects neither.

The proof points to the error. Both repairs begin from the rules the proofs name, and these rules are enough on their own: when a single wrong rule is hidden in an otherwise exact model, agents shown only a proof's rules find it in 29 of 42 runs, nearly as often as agents told which rule is wrong (30) and almost twice as often as agents shown per-rule evidence, while never flagging an error-free model (Appendix B.10).

5 Related work

Executable world models. Code world models are learned from interaction for planning (Tang et al., 2024), refined by generation, testing and repair (Dainese et al., 2024) or composed from programmatic experts (Piriyakulkij et al., 2025); theory-based agents explore and plan with learned theories (Tsividis et al., 2021), and TheoryCoder-2 learns reusable abstractions for hierarchical planning (Ahmed et al., 2026). Such models are central to recent ARC-AGI-3 agents (Rodionov, 2026a; Courtis et al., 2026; Rodionov, 2026b): Twin does not act until its simulator reproduces every observed transition (Skoutnev et al., 2026), and Tycho finds that a closer match does not by itself yield better play (Lehmann et al., 2026). The closest work, Memento 3, compiles a natural-language rulebook into code and accepts an update only if an LLM judges it faithful and replay is exact (Zhao et al., 2026). Yet a code world model can pass a sampled-transition check at 100% accuracy and still lose because it misses a key rule (Aguilar Martín, 2026). AWM instead abstracts its rules into a shared structure, tests them beyond the record, and proves reachability and impossibility, checking the proofs' premises against the game.

Certified impossibility. Unsolvability certificates let an independent checker confirm that a planning task has no plan (Eriksson et al., 2017), in the spirit of certifying algorithms (McConnell et al., 2011), and certificate checking has been formally verified for temporal planning (Wang & Abdulaziz, 2025); all start from a supplied model. Impossibility transfers from an abstraction only if it over-approximates the real system, budget semantics included (Clarke et al., 1994), and action-model learning gives conditions under which learned models do so (Aineto & Scala, 2024). AWM learns its model from interaction; Lean checks every deduction from that model, and interaction checks the model's rules against the game.

Formal reasoning. Neural provers produce Lean proofs at olympiad level (Achim et al., 2025; Chen et al., 2025; Hubert et al., 2026; Ren et al., 2025), AI systems have produced Lean proofs of open Erdős problems (Sothanaphan, 2026; Tsoukalas et al., 2026), and autoformalization translates informal statements into formal ones (Wu et al., 2022). There the objects of proof are given mathematics; AWM must first infer the dynamics and costs its proofs quantify over, which makes the fidelity of its formal model an empirical question.

6 Limitations and conclusion

Limitations. We demonstrate AWM on ARC-like environments with discrete actions and immediate, observable feedback, where learned rules can be checked against recorded transitions; partial observations, delayed feedback and stochastic dynamics are the next step. Lean guarantees the correctness of proofs under the learned model's explicit premises, and whether these premises capture the environment is tested by interaction: the explicit rules and kernel-checked proofs separate errors in the learned model from errors in reasoning, so a discrepancy can be traced to its assumption and addressed by further exploration. Broader environments and tasks are the natural next test of such formal reasoning.

Conclusion. Formal reasoning traditionally begins with predefined objects and rules; AWM discovers them through interaction, structures and simplifies them through category-theory-inspired transformations, and formalizes the result in Lean, where reachability and impossibility over action sequences never explored become kernel-checked theorems. On sealed levels, AWM proves in Lean what agents without it can only guess, and when such a proof is wrong, the rules it names locate the error in 29 of 42 runs.

Reproducibility statement

The readings of §2 and §3 come from the benchmark repository at commit `fa928185` (built 2026-10-24). Each game's model, tests and Lean project are regenerated by one command per game; the per-game Lean projects are checked with Lean 4.34.0 and do not use Mathlib. The readings of §4 come from the archive of the budget-sealed instance: both engine versions with their md5, the abstract model and its cross-check scripts with their raw outputs, the Lean files with their compile

logs and axiom audits, every file each agent wrote, and the verbatim end-of-attempt interviews; the per-run readings are given in the supplementary material. The game environments are third party; official replays are linked rather than redistributed.

Statement on AI use

The agents studied in this paper are LLM agents, so AI use is part of the method. Generative AI was also used to draft and edit the text and to write the analysis scripts. All reported numbers come from script outputs over the recorded runs, not from model-generated text. The Lean proofs are checked by the Lean kernel, independently of any model.

References

Tudor Achim et al. Aristotle: IMO-level automated theorem proving. *arXiv preprint arXiv:2510.01346*, 2025.

Javier Aguilar Martín. When a verified world model still loses: Play-adequacy vs prediction-accuracy in LLM-synthesized code world models. *arXiv preprint arXiv:2607.14169*, 2026.

Zergham Ahmed, Kazuki Irie, Joshua B. Tenenbaum, Christopher J. Bates, and Samuel J. Gershman. Learning abstractions for hierarchical planning in program-synthesis agents. *arXiv preprint arXiv:2602.00929*, 2026.

Diego Aineto and Enrico Scala. Action model learning with guarantees. In *Proceedings of the 21st International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pp. 801–811, 2024. doi: 10.24963/kr.2024/75.

ARC Prize Foundation. GPT-6 Astra: ARC-AGI results. `https://arcprize.org/results/openai-gpt-6-astra`, 2026a. Public Demo, 25 environments; accessed 2 October 2026.

ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026b.

Luoxin Chen, Jinming Gu, Liankai Huang, Wenhao Huang, Zhicheng Jiang, Allan Jie, Xiaoran Jin, Xing Jin, Chenggang Li, Kaijing Ma, et al. Seed-Prover: Deep and broad reasoning for automated theorem proving. *arXiv preprint arXiv:2507.23726*, 2025.

Edmund M. Clarke, Orna Grumberg, and David E. Long. Model checking and abstraction. *ACM Transactions on Programming Languages and Systems*, 16(5):1512–1542, 1994. doi: 10.1145/186025.186051.

David Courtis, Wenhao Li, and Scott Sanner. OPINE-World: Programmatic world modeling with ontology-error-prioritized interactive exploration for ARC-AGI-3. *arXiv preprint arXiv:2607.01531*, 2026.

Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, volume 37, pp. 60429–60474, 2024. doi: 10.52202/079017-1933.

Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pp. 625–635. Springer, 2021.

Salomé Eriksson, Gabriele Röger, and Malte Helmert. Unsolvability certificates for classical planning. In *Proceedings of the 27th International Conference on Automated Planning and Scheduling (ICAPS)*, pp. 88–97, 2017. doi: 10.1609/icaps.v27i1.13818.

Leonhard Euler. Solutio problematis ad geometriam situs pertinentis. *Commentarii academiae scientiarum Petropolitanae*, 8:128–140, 1741. Volume dated 1736; Eneström index E53.

Thomas Hubert, Rishi Mehta, Laurent Sartran, Miklós Z. Horváth, Goran Žužić, Eric Wieser, Aja Huang, Julian Schrittwieser, Yannick Schroecker, Hussain Masoom, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106):607–613, 2026. doi: 10.1038/s41586-025-09833-y.

Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.

Masaki Kashiwara and Pierre Schapira. *Categories and Sheaves*, volume 332 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 2006.

Jens Lehmann, Andrei Aioanei, and Sahar Vahdati. Tycho: Active abstraction with programmatic world models for ARC-AGI-3. *arXiv preprint arXiv:2607.28287*, 2026.

Saunders Mac Lane. *Categories for the Working Mathematician*, volume 5 of *Graduate Texts in Mathematics*. Springer, 2nd edition, 1998.

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi: 10.1016/j.cosrev.2010.09.009.

Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, Adrian Weller, Marta Kryven, and Kevin Ellis. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, volume 38, 2025. doi: 10.52202/085713-0896.

Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. *arXiv preprint arXiv:2504.21801*, 2025.

Sergey Rodionov. Executable world models for ARC-AGI-3 in the era of coding agents. In *Artificial General Intelligence: 19th International Conference, AGI 2026*, Lecture Notes in Computer Science. Springer, 2026a. arXiv:2605.05138.

Sergey Rodionov. Do coding agents need executable world models, simplification, and verification to solve ARC-AGI-3? *arXiv preprint arXiv:2607.15439*, 2026b.

Alexy Skoutnev, Kirill Acharya, Gaston Longhitano, Madeleine Udell, Kevin Ellis, and Iddo Drori. Twin: Playing an unknown game with a test-time digital twin. *arXiv preprint arXiv:2608.14490*, 2026.

Nat Sothanaphan. Resolution of Erdős problem #728: a writeup of Aristotle's Lean proof. *arXiv preprint arXiv:2601.07421*, 2026.

Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, volume 37, pp. 70148–70212, 2024. doi: 10.52202/079017-2243.

Pedro A. Tsividis, Joao Loula, Jake Burga, Nathan Foss, Andres Campero, Thomas Pouncy, Samuel J. Gershman, and Joshua B. Tenenbaum. Human-level reinforcement learning through theory-based modeling, exploration, and planning. *arXiv preprint arXiv:2107.12544*, 2021.

George Tsoukalas et al. Advancing mathematics research with AI-driven formal proof search. *arXiv preprint arXiv:2605.22763*, 2026.

David Wang and Mohammad Abdulaziz. Formally verified certification of unsolvability of temporal planning problems. *arXiv preprint arXiv:2510.10189*, 2025.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*, volume 35, 2022.

Yang Yuan. Succinct representations for concepts. *arXiv preprint arXiv:2303.00446*, 2023.

Haoyu Zhao, Zhengxu Yu, Zhiyuan He, Meng Fang, Rasul Tutunov, Haitham Bou-Ammar, Weilin Luo, and Jun Wang. Memento 3: Model-based recursive self-improvement through reflective rulebooks. *arXiv preprint arXiv:2610.11794*, 2026.